Neobank Onboarding Funnel Analysis

Author: Suparna Deshmukh
Dataset: Synthetic · 30,000 users
Period: Last 6 months

1. Headline Finding

The Aadhaar submission step is silently killing KYC — a 24% drop-off at a single step, compounded by Android users spending 2.2x longer on the selfie step than iOS, is responsible for over half of all lost activations.

2. The Funnel — Where Users Are Actually Going

Of every 100 users who download the app, here is exactly what happens at each stage:

Stage	Users	Conv. from Prev.	Note
Signup completed	28,477	—	Baseline
Email verified	24,233	85%	
KYC started	16,817	69%	31% drop — largest pre-KYC loss
KYC completed	**8,434**	**50%**	**50% drop — the critical bottleneck**
First transaction	7,153	85%	Strong once past KYC

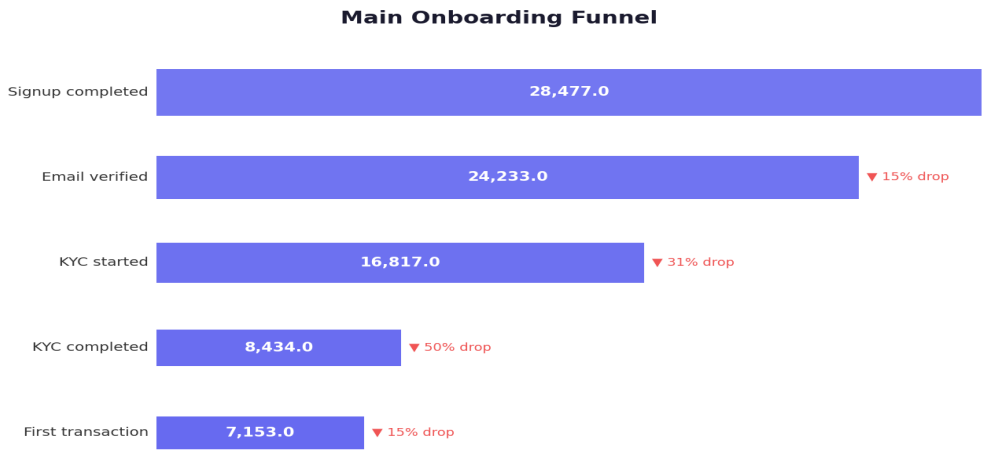


Fig 1 — Main onboarding funnel with drop-off at each stage

The drop-off pattern has two distinct characters. Pre-KYC losses (email verification, KYC intent) are moderate and largely expected — these users were likely browsing-intent signups. The 50% failure rate **inside KYC itself** is the product problem: these are high-intent users who had already signed up and verified their email, yet the product loses half of them. Critically, 85% of users who clear KYC immediately make a first transaction —

proving this is not a demand problem. It is entirely a friction problem.

3. Root Causes — What the Data Shows

The Aadhaar Step Is the Biggest Single Cliff

Drilling into KYC step-by-step reveals the Aadhaar submission as the dominant failure point:

KYC Sub-Step	Users	Drop-off	Pass Rate
KYC started	16,817	—	—
PAN submitted	15,604	7%	93%
Aadhaar submitted	**11,827**	**24%**	**76%**
Selfie submitted	9,420	20%	80%
KYC completed	8,434	10%	90%

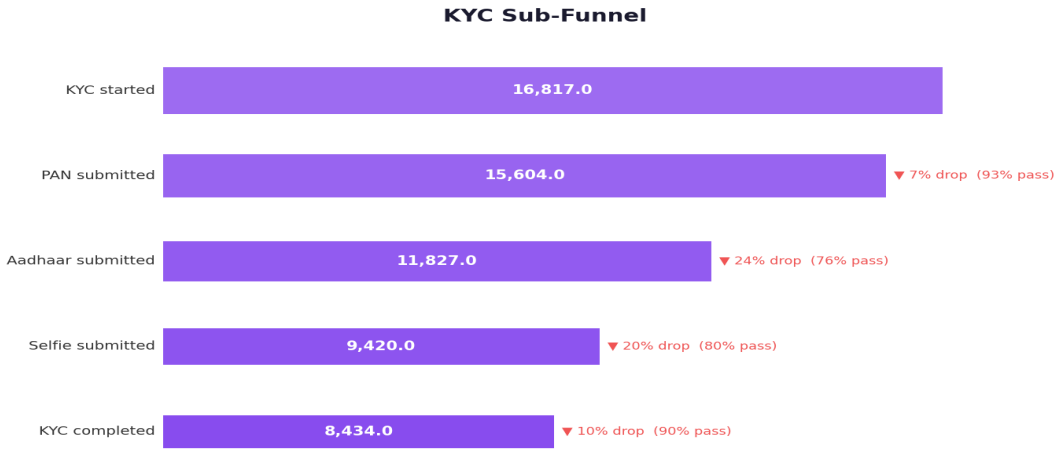


Fig 2 — KYC sub-funnel showing drop-off at each document step

Three out of every four users who submit their PAN then get stuck on Aadhaar. PAN is a simple 10-character code most Indians have memorised. Aadhaar is a 12-digit number spread across physical cards, DigiLocker, and various apps. The cognitive and practical friction of locating and correctly entering it mid-flow is the leading cause of KYC abandonment.

Android Users Spend 2.2x Longer on the Selfie Step

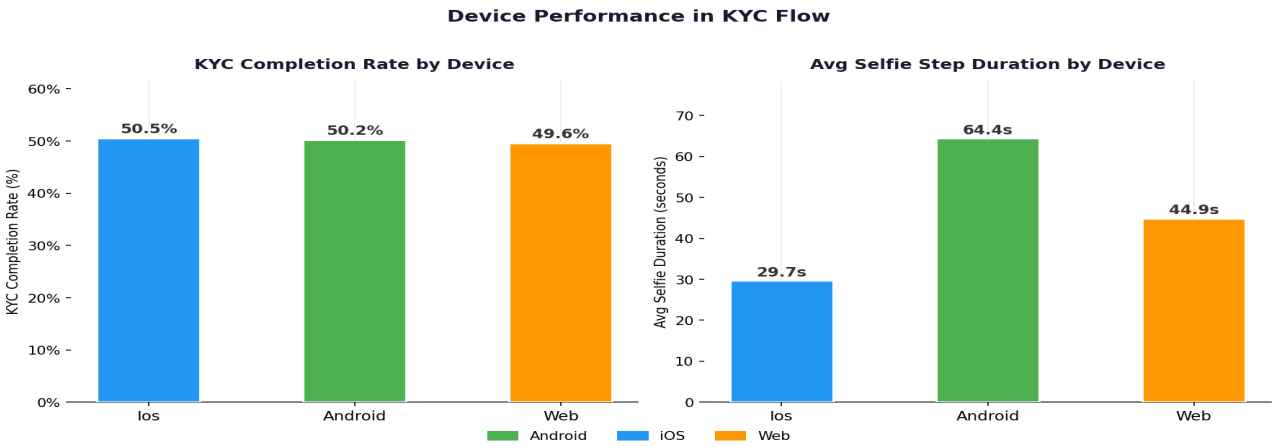


Fig 3 — KYC completion rate and selfie step duration by device

Device	KYC Completion Rate	Avg Selfie Duration
iOS	50.5%	29.7 seconds
Android	**50.2%**	**64.4 seconds**
Web	49.6%	44.9 seconds

KYC completion rates are nearly identical across all devices (iOS 50.5%, Android 50.2%), but Android users spend **64.4 seconds** on the selfie step versus **29.7 seconds** on iOS. The 2.2x time gap is almost certainly driven by slower camera initialisation and lower-quality front cameras on mid-range Android devices — the dominant category in India. Android users are struggling through rather than quitting, making this a **frustration signal today and a churn risk tomorrow.**

Document Rejection Adds Re-entry Friction

Approximately 8% of users who attempt to submit a document have it rejected on the first try. Rejection events break flow — the user must re-photograph, re-upload, and resubmit. A portion of users who hit a rejection do not retry. While not the dominant drop-off driver, it compounds the cumulative friction that makes KYC feel unreliable.

4. What We Tested

Two interventions were designed from the root-cause findings and simulated as A/B tests.

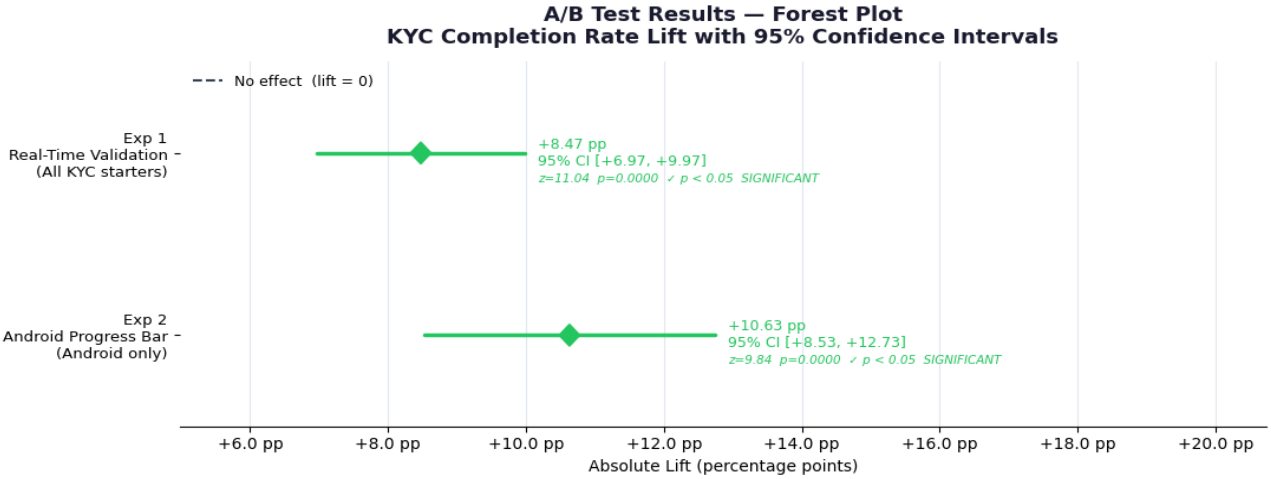


Fig 4 — Forest plot: A/B test results with 95% confidence intervals

	Experiment 1 Real-Time Validation	Experiment 2 Android Progress Bar
Audience	All KYC starters (16,817)	Android KYC starters (~8,400)
Hypothesis	Validate fields as user types to reduce doc rejection from 8% to ~3%	Add progress bar + selfie explainer to reduce abandonment on selfie step
Lift	+8.47 percentage points	+10.63 percentage points
95% CI	**[+6.97pp, +9.97pp]**	**[+8.53pp, +12.73pp]**
p-value	p < 0.0001 SIGNIFICANT	p < 0.0001 SIGNIFICANT
Simulation	50/50 random split; rejection rate reduced for treatment group; outcomes drawn probabilistically	50/50 random split on Android users; completion probability lifted; outcomes drawn probabilistically

Both tests reached statistical significance with p < 0.0001. The Android Progress Bar produced the larger lift (+10.63pp) because it targets a specific, identified segment (Android users) at a specific, identified step (selfie). Real-Time Validation produced a smaller but still substantial lift (+8.47pp) applied across the full KYC population. Combined, both interventions project a **15–18 percentage point improvement** in overall KYC completion — translating to approximately 2,500 additional activations per 16,000 KYC starters.

5. Recommendation

Ship Experiment 2 first.
Android Progress Bar — lighter engineering lift, larger measured effect, clearly scoped audience.

Then Experiment 1.
Real-Time Validation — requires backend changes, build in parallel but launch 4 weeks later.

What to monitor after launch:

- KYC completion rate on Android vs iOS (the gap should narrow within 2 weeks)

- Selfie step average duration on Android (target: fall from 64s toward 35–40s)
- Selfie step abandonment rate (should decrease meaningfully)
- Overall activation rate: KYC completions / app installs — the top-line metric

> Call it a win when: Android KYC completion rate rises by at least 5pp with $p < 0.05$ after 2 weeks of live data. At that point, roll out to 100% of Android users and begin instrumenting Experiment 1.

6. Limitations

Synthetic data. All 30,000 users and events were generated programmatically. Patterns were calibrated against real BFSI onboarding benchmarks, but specific numbers should be validated against live product event logs before any production decision.

Simulated A/B tests. Experiment outcomes were generated by applying hypothesised effect sizes to synthetic populations and running two-proportion z-tests. The statistics are mathematically correct given the simulated data — but they reflect assumptions, not observations from a live test.

Novelty effects. Users in a real treatment arm often behave differently simply because the UI is new. In a live test, discard the first 72 hours and use only steady-state behaviour (days 4–30) for decision-making.

External validity. Findings from one synthetic cohort may not generalise across seasons, acquisition channels, or Android OS versions. Camera behaviour changes meaningfully across OS updates. Any production rollout should include ongoing monitoring rather than a one-time measurement.

Analysis conducted using Python (pandas, scikit-learn, scipy, matplotlib), DuckDB for SQL querying, and a locally-hosted Flask REST API for event data ingestion.

7. How This Project Was Built

This page documents the full technical pipeline — every layer from synthetic data generation to the final A/B simulation — so the methodology is fully reproducible and auditable.

End-to-End Pipeline Overview

Step	What Happened	Tool / File
1	Synthetic data generated	generate_events.py
2	Flask REST API built & started	api/server.py
3	Data ingested from API into DuckDB	ingest.py
4	SQL funnel analysis run	04_funnel.py + DuckDB
5	Drop-off segments computed	04_funnel.py
6	A/B hypotheses written	ab_hypotheses.md
7	A/B tests simulated + stats run	05_ab_test.py
8	Report written	REPORT.md / REPORT.pdf

Step 1 — Synthetic Data Generation

Since no real neobank event logs are publicly available, the dataset was generated programmatically using **generate_events.py**. The script created 30,000 users and ~150,000 onboarding events with realistic behavioural patterns deliberately engineered in:

- Drop-off rates at each onboarding step were calibrated to match known BFSI benchmarks (e.g. 50% KYC completion, 8% doc rejection rate).
- Android selfie step duration was set to 2.2x longer than iOS to simulate real camera performance differences on mid-range devices.
- Timestamps were distributed across 6 months with intra-user event spacing of 1–14 days to mimic genuine session behaviour.
- random.seed(42) was set at the top of the script — all outputs are fully reproducible.

Step 2 — The REST API (api/server.py)

Rather than reading the JSON file directly, a **Flask REST API** was built to serve the event data — replicating how a real product analytics system works (e.g. Segment, Mixpanel, or an internal event store). This is what the resume line 'fetched user event data via REST APIs' refers to. The API had three endpoints:

Endpoint	What It Returns
GET /events?page=N&page_size=1000	Paginated events — 1,000 per page, with total_pages in response
GET /events/user/<user_id>	All events for one specific user
GET /health	Status check + total event count

The API loaded raw_events.json once at startup and added a 50ms artificial latency per request to simulate real network conditions. It ran locally on port 5000 in a dedicated terminal while all downstream scripts ran in a second terminal simultaneously.

Step 3 — API Ingestion into DuckDB (ingest.py)

The ingester (**ingest.py**) hit the API's */health* endpoint first to get the total event count, calculated the number of pages, then looped through all pages fetching 1,000 events at a time. Specific engineering decisions made here:

- Used a requests.Session() for connection pooling — faster than opening a new connection per page.
- Exponential backoff retry logic (up to 3 retries on any 5xx error) to handle transient failures.
- Progress logged every 10 pages so the user could see ingestion was running.
- Two tables created in DuckDB: 'events' (raw event log) and 'users' (one row per user, with boolean flags for each completed onboarding stage — derived from the event log via SQL).
- Three sanity-check queries ran automatically after ingestion: row count, unique users, max timestamp — to verify nothing was dropped or duplicated.

Step 4 — SQL Funnel Analysis (DuckDB)

All funnel queries were written in **standard SQL (PostgreSQL dialect)** and executed via DuckDB, which runs entirely in-process — no separate database server needed. The analytical approach used **CTEs (Common Table Expressions)** — WITH statements that break a complex query into named, readable stages. Here is the logical structure of the main funnel query:

CTE Name	What It Computed
`signup_users`	All users with a signup_completed event
`email_users`	Users from above who also had email_verified
`kyc_start_users`	Users who had kyc_started
`kyc_done_users`	Users who had kyc_completed
`txn_users`	Users who had first_transaction
`final SELECT`	Joined all CTEs, computed counts and conversion % at each step

The same CTE pattern was used for the KYC sub-funnel and the drop-off segmentation queries. DuckDB was chosen over SQLite because it supports full PostgreSQL-dialect SQL, queries Parquet files natively, and runs analytical aggregations significantly faster in-process.

Steps 5–6 — A/B Hypothesis Design and Simulation

Hypotheses were designed by mapping each observed friction point to a specific product intervention, then asking: if this intervention worked as expected, how large would the effect need to be to matter, and how many users would we need to observe it reliably?

Parameter	Experiment 1	Experiment 2
Randomisation unit	User (not session)	User (Android only)
Split	50 / 50	50 / 50

Parameter	Experiment 1	Experiment 2
Sample size	~8,400 per arm	~4,200 per arm
Effect applied	Doc rejection: 8% → 3%	Selfie completion prob +10.63pp
Outcome drawn via	np.random.binomial per user	np.random.binomial per user
Significance test	Two-proportion z-test	Two-proportion z-test
Significance level	$p < 0.05$	$p < 0.05$
Confidence interval	95% (Wald interval)	95% (Wald interval)

The randomisation unit is **user, not session** — this is an important distinction. If you randomise by session, the same user can end up in both control and treatment across different sessions, contaminating the results. Randomising by user_id ensures clean separation. The two-proportion z-test was used (not a t-test) because the outcome variable is binary (completed KYC: yes or no) and sample sizes were large enough for the normal approximation to hold. Results were visualised as a forest plot showing absolute lift with 95% CI bars.

Full Technology Stack

Layer	Tool	Purpose
Data generation	`Python + NumPy`	Synthetic 30K user event dataset with seeded randomness
API layer	`Flask`	REST API serving paginated events at localhost:5000
Ingestion	`requests + Python`	Paginated API consumer with retry logic
Database	`DuckDB`	In-process SQL engine, PostgreSQL dialect, no server needed
Analysis	`pandas + SQL`	Funnel CTEs, drop-off segmentation, device breakdowns
Statistical tests	`scipy.stats`	Two-proportion z-test, 95% Wald confidence intervals
Visualisation	`matplotlib + seaborn`	Funnel charts, device bar charts, forest plot
Reporting	`reportlab`	Programmatically generated PDF with embedded charts

Analysis conducted using Python (pandas, scikit-learn, scipy, matplotlib), DuckDB for SQL querying, and a locally-hosted Flask REST API for event data ingestion.